MEDNOVA

SMART DIGITAL HEALTHCARE SaaS PLATFORM

Final Year Project Defense Guide

& Comprehensive Technical Documentation

Project Focus: Multi-Tenant SaaS, Smart Scheduling, Blood Bank Network & Diet Planners

Architecture: Custom PHP MVC (Framework-Free), MySQL ACID Transactions

Frontend: Asynchronous jQuery, CSS Variables, Responsive Grid Layouts

Document Includes:

Environment Prep: Local XAMPP/MariaDB Auto-Probing Setup & Recovery

Prepared for final presentation before external examiners and professors.

Generated on June 2, 2026

Welcome to the official **MedNova** Defense Preparation and Technical Documentation Guide. This document has been prepared specifically for your team to study, prepare, and confidently present your Final Year Project (FYP) to external examiners and academic professors.

1. Project Overview

1.1 What is MedNova?

MedNova is a multi-tenant, role-based, software-as-a-service (SaaS) healthcare orchestration platform designed to digitize clinical operations, patient scheduling, dietitian mapping, and blood donor networks in regional healthcare ecosystems.

1.2 What Problem Does It Solve?

Regional districts (such as Kohat) often suffer from highly fragmented, manual healthcare operations:

- **Appointment Bottlenecks**: Patients stand in long physical lines for tokens, or doctors face unorganized, overlapping daily queues.
- **Information Silos**: Patient history, medical prescriptions, and diet plans are scattered across paper files.
- **Emergency Response Delays**: Finding local blood donors of a specific type quickly during emergency trauma requires manual phone calls and social media broadcasting.
- **SaaS Deficit**: Smaller clinics cannot afford enterprise-grade hospital software, while large systems lack localized modularity.

MedNova bridges this gap by offering a single, unified database system that dynamically scopes operations based on active login roles and hospital subscriptions.

1.3 Main Users of the System

The system is divided into five distinct access control roles:

1. **Super Admin (Platform Owner)**: Oversees subscription plans, edits hospital registrations, configures global system settings, and reviews system-wide stats.
2. **Hospital Admin**: Scopes and configures a specific clinic/hospital. Manages local departments, schedules, doctors, receptionists, and reviews analytical statistics.
3. **Doctor**: Reviews daily patient queues, executes patient diagnoses, records prescriptions, prints A4 documents, and issues custom diet plans.
4. **Receptionist**: Registers walk-in patients, assigns them to active doctors, and prints physical token slips.
5. **Patient**: Registers online, searches for facilities and doctors by region, requests appointments, views history, requests diet plans, and broadcasts emergency blood requests.

1.4 Main Modules

- **Clinic & Hospital SaaS Manager**: Multi-tenant facility profiles, region filter mappings, and dashboard analytics.
- **Smart Scheduling & Booking**: Auto-token ordering, multi-step booking forms, and dynamic calendar slots.
- **Digital Prescription & EMR**: Diagnosis tracker, text-based medicine records, and print-ready prescriptions.
- **Blood Bank Registry**: Real-time donor registry and broadcast notifications of urgent blood requests.

- **Smart Diet Planner**: Nutritional goal selection (weight loss, muscle gain) and dietitian prescription forms.
- **Self-Service Health Assessment Suite**: Clients can run BMI checkups, daily calorie limits, water intake targets, and blood donation eligibility checks.

1.5 Technologies Used

- **Frontend**: HTML5, Vanilla CSS3 (custom CSS design systems, animations, glassmorphism), JavaScript (ES6+), jQuery, Bootstrap 5.3.
- **Backend**: PHP 8.x (Clean, modular OOP MVC architecture without external frameworks).
- **Database**: MySQL 8.x / MariaDB (custom normalization, transactions).
- **Security & Libraries**: BCrypt password hashing, session tokens, PHPMailer (SMTP configuration).

1.6 Value Proposition for Healthcare Institutions

- **Zero Infrastructure Overhead**: Standard XAMPP/WAMP or Hostinger cloud compatibility.
- **Auto-Discovery Database Probing**: Detects ports automatically (`3306`, `3308`, `3307`), auto-creates databases, and auto-imports SQL schemas on setup.
- **Dynamic Modularity**: Easily toggles components depending on hospital plans.

---

2. Complete Frontend Workflow

The frontend of MedNova is built to provide an interactive, animated, responsive experience that connects with the backend API endpoints asynchronously without reloading the page.

2.1 Core Architectural Flow

```
User Opens Website
        ↓
HTML Page Loads (DOM tree built)
        ↓
CSS Styles the Page (Applies custom themes, animations, glassmorphism)
        ↓
JavaScript / jQuery Initializes (Attaches event listeners, fetches initial dropdown values)
        ↓
User Fills out and Submits Form
        ↓
JavaScript Intercepts Submit (Validates inputs, formats data as JSON payload)
        ↓
Asynchronous fetch() Request (Sent to backend API: backend/index.php?route=api/...)
        ↓
Backend Processes & Returns Response (Sends JSON response with success/fail status)
        ↓
JavaScript Modifies DOM (Updates UI states, shows success panels, updates patient tables)
```

2.2 Key Frontend Files

- **HTML Pages (`frontend/*.html`): Structure content (About, Services, Doctors, Contact, Blood Donation, Diet Plan, Emergency Triage, Emergency SOS, Health Assessment, etc.).**
- **CSS Stylesheet (`frontend/css/style.css`)**: Implements modern typography, color palette, custom cards, CSS reveal animations (`mn-reveal`), and layout variables.
- **JavaScript Engine (`frontend/js/app.js`)**: Grouped into six core components:

1. *API Service Layer*: Intercepts and performs async requests to `/regions`, `/hospitals`, `/doctors`, `/departments`, `/appointment`, etc.
2. *Global UI Enhancements*: Handles navbar styling, dynamic back-to-top button injection, smooth anchor scrolls, and ripple effects.
3. *Regional Data Sync*: Cascading dropdown chains (selecting a Region queries Hospitals, selecting a Hospital queries Departments, which queries Doctors).
4. *Component Search*: Filters clinical facilities and displays details on sliding panels.
5. *Form Submission handlers*: Collects, validates, and transmits payloads.
6. *Blogs Engine*: Dynamically queries and parses recent RSS-like blog feeds.

3. Complete Backend Workflow

The backend operates under a lightweight Model-View-Controller (MVC) pattern written in clean, raw PHP, ensuring fast performance and easy tracing during exams.

3.1 Core Backend Flow

```
Frontend Request (GET/POST /?route=controller/method)
        ↓
Backend Entry Point (backend/index.php)
        ↓
Router (backend/core/Router.php: Parses query string, maps to controller class & method)
        ↓
Controller (backend/controllers/XController.php: Enforces session role controls, executes logic)
        ↓
Model / Database Query (backend/models/X.php: Prepares SQL statement, executes transaction)
        ↓
Database (MySQL/MariaDB processes query and returns matching rows)
        ↓
Controller Processes Data (Maps results, loads parameters, handles email triggers)
        ↓
Response Returned (Sends either JSON payload to API calls, or extracts variables and renders PHP views)
        ↓
Frontend Renders Result (Displays dashboards, listings, or error banners)
```

3.2 Key Backend Architecture Files

- **Entry Point (`backend/index.php`)**: Bootstraps sessions, includes core dependencies, and initiates the routing dispatcher.
- **Router (`backend/core/Router.php`)**: Dynamically resolves strings like `auth/login` to `AuthController` and the `login()` method.
- **Base Controller (`backend/core/Controller.php`)**: Instantiates PDO database handles, renders HTML templates by extracting data variables, and contains helper wrappers for SMTP emailing.
- **Database Manager (`backend/config/Database.php`)**: Auto-probes active database ports, auto-creates database schemas, and holds connection keys.

4. Module-by-Module Workflow

4.1 Authentication Module

- **Login Flow**: Patient or Administrator inputs email/phone and password -> POST request sent to `?route=auth/login` -> Controller verifies username/email against database -> Runs `password_verify()` against salted BCrypt hash -> Restricts action if account is inactive -> Initializes `$_SESSION['user_id']`, `$_SESSION['role']`, `$_SESSION['name']`, `$_SESSION['email']` -> Redirects user based on role:
- `admin` -> `?route=admin/dashboard`
- `hospital_admin` -> `?route=hospital_admin/dashboard`
- `doctor` -> `?route=doctor/dashboard`
- `receptionist` -> `?route=receptionist/dashboard`
- `patient` -> `?route=patient/dashboard`
- **Logout Flow**: User clicks logout -> Request sent to `?route=auth/logout` -> Session cleared via `session_unset()` and destroyed via `session_destroy()` -> User redirected to `?route=auth/login`.

4.2 Appointment Module

- **Booking Flow**: Patient chooses Region -> selects Hospital -> selects Department -> selects Doctor -> enters Date/Time -> submits booking form.
- **Backend Processing**: `ApiController::appointment()` intercepts request inside a transaction -> registers user as patient if new -> counts existing bookings for the day to generate next `token_number` -> inserts appointment with `status = 'pending'` -> triggers in-app notification for the hospital -> commits transaction.
- **Dashboard Verification**: Hospital Admin views the booking request -> clicks `Confirm` -> status updates to `confirmed`.
- **Doctor Checkup**: Active appointment appears in the Doctor's daily Patient Queue -> Doctor clicks `Checkup` -> records diagnosis/prescriptions -> submits -> status updates to `completed`.

4.3 Doctors & Hospitals Module

- **Data Flow**: Core data is fetched dynamically. JavaScript queries `api.php?route=api/regions` to fill regional dropdowns. Once a region is chosen, it queries `/hospitals?region_id=X` to fetch only subscribed facilities in that region. When a hospital is selected, departments are queried, which dynamically filters the active doctors.
- **Database Relationships**:
- One Region has many Hospitals (`hospitals.region_id` -> `regions.id`).
- One Hospital has many Doctors through a junction table (`doctor_hospital_affiliations`).
- One Doctor has details (specialty, experience, fee) stored in `doctors_details`.

4.4 Blood Donation Module

- **Donor Registration**: User registers as a donor by providing age, blood type, contact details, and selecting a local

hospital database base.

- **Blood Request Creation**: A patient or hospital admin logs a request for a specific blood group, indicating the level of urgency (critical, normal) and contact info.
- **Notification Broadcast**: The system records the request and automatically publishes it. An in-app alert is broadcasted to the hospital dashboard, and matching blood groups are filtered across the local donor database.

4.5 Diet Plan Module

- **Creation Flow**: Doctor opens patient card -> fills out diet plan parameters (Daily Caloric target, Protein/Carbohydrates/Fats distribution, Meal schedules, and custom instructions) -> links patient user ID -> submits -> saves to `diet_plans` table in database.
- **Patient Access**: Patient logs into their dashboard -> clicks `Diet Plan` -> reads active plan and calorie limits.

4.6 Health Assessment Module

- **BMI Calculator**: Patient inputs Weight (kg) and Height (cm).
- *Formula*: $BMI = \frac{Weight}{(Height/100)^2}$
- *Categories*: Underweight (<18.5), Normal (18.5-24.9), Overweight (25-29.9), Obese (>=30).
- **Daily Calorie Calculator**: Inputs Weight, Height, Age, Gender, and Activity Level.
- *Mifflin-St Jeor Formula*:
- Men: $BMR = 10 \times Weight + 6.25 \times Height - 5 \times Age + 5$
- Women: $BMR = 10 \times Weight + 6.25 \times Height - 5 \times Age - 161$
- Multiplies BMR by activity multiplier (Sedentary: 1.2, Light: 1.375, Moderate: 1.55, Active: 1.725) to return daily maintenance calories.
- **Water Intake Calculator**: Inputs Weight (kg) and Activity Level.
- *Formula*: $Water (ml) = (Weight \times 35) + (Activity \times 500)$ where Activity is hours of exercise.
- **Donation Eligibility Checker**: Validates Age (18-65), Weight (>=50kg), Last Donation Date (>90 days), and general health checkbox queries to confirm eligibility.

4.7 Admin Dashboard (Super Admin)

- **Features**: System statistics (total hospitals, active subscription revenue, doctor accounts count), hospital profiles manager (edit details, toggle subscription status, extend plan durations), settings adjustments, and database backups (generating raw `.sql` file downloads and restoring schema).

4.8 Doctor Dashboard

- **Features**: Daily checkup queue based on current date, waiting patients counter, patient diagnosis drawer (recording symptoms, prescription, advice), and printable prescriptions.

4.9 Patient Dashboard

- **Features**: Recent appointments timeline list, assigned diet plan view, blood request manager, and access to saved

self-assessment reports.

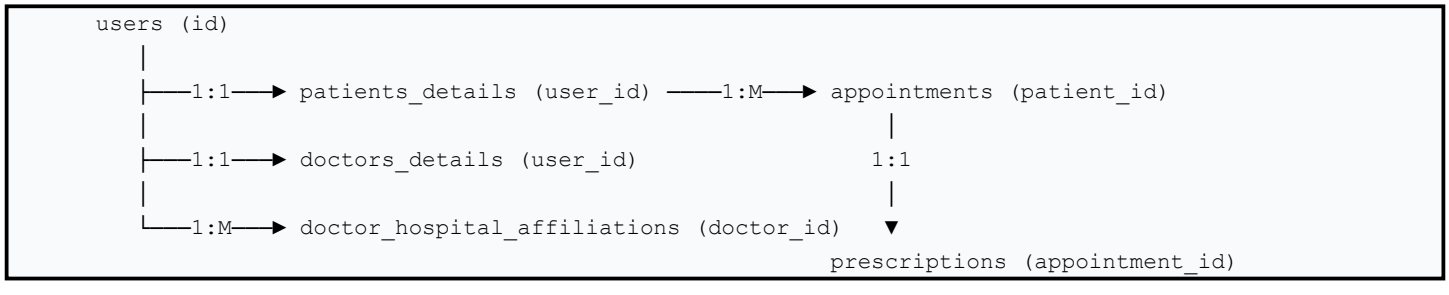
5. Database Workflow

5.1 Main Database Tables

The database consists of normalized tables linked through primary and foreign keys:

Table	Purpose	Primary Key	Foreign Key / Links
users	Holds login credentials for all roles	`id`	None
hospitals	Stores registered hospital metadata	`id`	`user_id` -> `users.id`
departments	Clinical departments at a hospital	`id`	`hospital_id` -> `hospitals.id`
doctors_details	Specialization and experience details	`id`	`user_id` -> `users.id`
doctor_hospital_affiliations	Junction table mapping doctors to facilities	`id`	`doctor_id` -> `users.id`, `hospital_id` -> `hospitals.id`
patients_details	Profile card for registered patients	`id`	`user_id` -> `users.id`
appointments	Log of patient appointments	`id`	`patient_id` -> `patients_details.id`, `doctor_id` -> `users.id`, `hospital_id` -> `hospitals.id`
prescriptions	Medical details written by doctors	`id`	`appointment_id` -> `appointments.id`
diet_plans	Assigned diet routines	`id`	`patient_id` -> `users.id`, `doctor_id` -> `users.id`
blood_donors	Registered blood donors	`id`	`user_id` -> `users.id`, `hospital_id` -> `hospitals.id`
blood_requests	Logs urgent blood needs	`id`	`hospital_id` -> `hospitals.id`
notifications	Logs active system events	`id`	`hospital_id` -> `hospitals.id`

5.2 Table Relationships



5.3 Normalization

- **1NF**: Atomic values (no multiple phone numbers in one field; emails are unique).
- **2NF**: No partial dependencies (all non-key fields depend entirely on the primary key).
- **3NF**: No transitive dependencies (e.g., doctor department is mapped through a department ID, not stored as plain text inside the doctor details table).

6. API Documentation

All frontend modules communicate with the backend through REST-style JSON endpoints.

Route	Method	Purpose	Input Parameters	Output Format	Called By Page
api/regions	GET	Fetch all system regions	None	Array of Region Objects	`appointment.html`, `diet_plan.html`, `blood-donation.html`
api/hospitals	GET	Fetch active hospitals	`region_id`, `type`, `search`	Array of Hospital Objects	`appointment.html`, `diet_plan.html`, `blood-donation.html`
api/doctors	GET	Fetch doctors	`hospital_id`	Array of Doctor Objects	`appointment.html`, `diet_plan.html`
api/departments	GET	Fetch hospital departments	`hospital_id`	Array of Department Objects	`diet_plan.html`
api/appointment	POST	Book patient appointment	`name`, `phone`, `date`, `time`, `hospitalId`, `doctorId`	JSON Success/Error	`appointment.html`
api/dietPlan	POST	Request smart diet plan	`name`, `phone`, `age`, `weight`, `height`, `hospitalId`, `doctorId`, `goal`	JSON Success/Error	`diet_plan.html`
api/registerDonor	POST	Register blood donor	`name`, `age`, `blood_group`, `phone`, `location`, `hospitalId`	JSON Success/Error	`blood-donation.html` (Donor form)
api/requestBlood	POST	Broadcast blood request	`patient_name`, `blood_group`, `urgency`, `phone`, `hospitalId`	JSON Success/Error	`blood-donation.html` (Patient form)
api/contact	POST	Submit contact message	`name`, `phone`, `message`	JSON Success/Error	`contact.html`

7. File Structure Explanation

```
MedNova/
│
├── backend/                  # CORE BACKEND SYSTEM
│   ├── config/
│   │   └── Database.php      # Database connection, port prober & schema loader
│   ├── core/
│   │   ├── Router.php        # URL parsing & controller dispatch engine
│   │   └── Controller.php    # Base controller (view rendering, SMTP configuration)
│   ├── controllers/
│   │   ├── AuthController.php # Login, logout & session mapping
│   │   ├── ApiController.php  # Handles REST requests and returns JSON payloads
│   │   ├── AdminController.php # Super admin analytics, CRUD, and SQL backups
│   │   └── HospitalAdminController.php # Scoped stats, live alerts, settings
│   ├── models/
│   │   └── Appointment.php   # Appointment querying & token counters
│   ├── views/                # VIEW templates
│   │   ├── layouts/
│   │   │   ├── header.php    # Dashboard header, notifications lists
│   │   │   └── sidebar.php   # Left menu sidebar links
│   │   ├── auth/             # Login view template
│   │   ├── admin/            # Super Admin panel templates
│   │   ├── hospital_admin/   # Hospital Admin panel templates
│   │   └── doctor/           # Doctor panel templates
│   ├── index.php             # Core app bootstrap entry point
│   ├── schema.sql            # Master database structure
│   └── seed_dummy.php        # Dummy database data seeder
│
└── frontend/                 # USER LANDING PORTAL
    ├── css/
    │   └── style.css         # Styling system & animations
    ├── js/
    │   └── app.js            # Asynchronous handlers & DOM controllers
    ├── index.html            # Portal Home
    ├── appointment.html      # Appointment Booking
    ├── diet_plan.html        # Diet plan requests
    ├── blood-donation.html   # Blood bank interface
    └── health-assessment.html # Assessment suit
```

8. Defense Presentation Guide

Use these scripts during your presentation before the evaluation panel.

8.1 2-Minute Elevator Pitch

> *"Good morning, respected examiners. Today, we present **MedNova**, a multi-tenant SaaS healthcare platform. Our system addresses the severe operational inefficiencies and lack of digitization in regional health networks. MedNova operates under a custom Model-View-Controller (MVC) architecture built entirely in PHP and MySQL. It features a Smart Appointment module with queue token generation, a real-time Blood Donation registry with active broadcast alerts, an automated Diet Planner based on user metrics, and a patient Self-Assessment suite. By offering multi-tenancy, it allows individual clinics and hospitals in regional areas like Kohat to register, configure departments, and manage their clinical workflows on a unified cloud interface without expensive infrastructure overhead. Thank you."*

8.2 5-Minute Summary Script

> *"Respected panel, our project, MedNova, is designed to bring premium digital healthcare management to regional areas. We have developed this system using a three-tier MVC architecture.*
>
> ***First**, on the presentation layer, the frontend uses custom CSS design systems and an asynchronous JavaScript API layer (`app.js`) to provide smooth, single-page interactions.*
>
> ***Second**, our logic layer, built on PHP OOP, features a custom URL router (`Router.php`) that parses incoming routes and dispatches them to appropriate controllers, preventing dependencies on heavy frameworks.*
>
> ***Third**, our database layer in MySQL features strict relational schemas that prevent data redundancy.*
>
> *During this demo, we will guide you through the patient flow-showing how they search clinics, book appointments, and access health assessment calculators. Then, we will show the receptionist interface for walk-in management, the doctor panel for digital prescriptions, and finally, the hospital admin dashboard which aggregates scoped stats. MedNova's unique selling point is its database resilience-automatically detecting database ports, self-creating structures, and importing SQL seeds on setup, making it highly portable. Let us now begin the live demonstration."*

8.3 10-Minute Detailed Presentation Script

- **Slide 1: Title Slide (MedNova)**

> *"Respected examiners, we welcome you to our final presentation of MedNova. Our team has built this system to digitize and connect regional healthcare clinics under a single SaaS system."*

- **Slide 2: Problem Statement & Objectives**

> *"In regional health sectors, operations are highly manual. Patient booking is disorganized, blood donors cannot be matched quickly in emergencies, and smaller clinics lack the budget for large ERP systems. MedNova's objective is to solve this by providing a scalable, low-overhead multi-tenant solution."*

- **Slide 3: Architectural Design**

> *"We avoided heavy frameworks to keep the runtime lightweight. We built our own MVC engine. Requests go to `index.php`, where our `Router` resolves them. Controllers handle authentication and sessions, query data via Models, and render views dynamically. The frontend communicates with the controllers via asynchronous JSON APIs."*

- **Slide 4: Database Schema & Integrity**

"Our database features 21 normalized tables. We use strict foreign key constraints. In critical flows like appointment booking and doctor creation, we use database transactions (`beginTransaction()`, `commit()`, `rollBack()`) to prevent data corruption."

- **Slide 5: Core Module 1 (Smart Appointments)**
  "Patients book appointments by choosing regions, hospitals, and doctors. The system counts existing bookings for that doctor and date to generate a queue token number. This ensures a transparent, first-come-first-served checkup flow."

- **Slide 6: Core Module 2 (Prescriptions & EMR)**
  "Once an appointment is confirmed, the doctor can write a digital prescription. The prescription data is linked to the appointment, and the system generates an A4-print-ready prescription page for the patient."

- **Slide 7: Core Module 3 (Blood Bank & SOS Alerts)**
  "The system includes a donor registry. When a critical patient needs blood, the request is broadcasted. A notification is sent to the local hospital admin, and donors with matching blood groups are instantly visible."

- **Slide 8: Core Module 4 (Health Assessments & Diet Plans)**
  "We have built custom health calculators. Patients calculate their BMI, calories, and water intake, and receive target metrics. Doctors can review these metrics and assign custom diet plans directly to the patient's portal."

- **Slide 9: System Demo & Implementation**
  "Let us now walk you through the live system, showing the interactions across all user roles."

- **Slide 10: Conclusion & Future Scope**
  "MedNova is fully functional. In the future, we plan to integrate SMS gateways for real-time mobile reminders, add optical character recognition (OCR) to parse medical lab reports automatically, and introduce predictive AI for doctor recommendations. We are now open for questions."

9. Demo Flow for Final Defense

Follow this sequential demo order to showcase MedNova's capabilities:

Step 1: Portal Homepage

- **What to Click**: Load `http://localhost/FinalFYP/frontend/index.html`. Scroll down to show sections.
- **What to Say**: *"This is our landing page. It showcases active statistics (200+ patients, 20+ service areas), quick links, and services. It uses custom CSS variables to support fluid glassmorphism design styles."*
- **Demonstrated**: Premium UI layout, responsiveness, and blogs feed engine.

Step 2: Navbar & Dynamic Dropdowns

- **What to Click**: Click on the **Smart Features** dropdown. Click **Smart Diet Plan**.
- **What to Say**: *"The navbar dynamically adapts. When we select a region, JavaScript calls our backend API in the background to fetch and populate only the hospitals inside that region. Selecting a hospital then filters its departments and doctors. This eliminates invalid selections."*
- **Demonstrated**: Asynchronous dropdown data synchronization.

Step 3: Health Assessment Tools

- **What to Click**: Go to **Health Assessment** dropdown. Click **BMI Calculator**. Input weight: `70`, height: `175`. Click calculate.
- **What to Say**: *"Our system includes self-assessment calculators. Here, we calculate BMI. The page computes the score, displays the category (Normal), and suggests matching dietary advice without needing a page refresh."*
- **Demonstrated**: Health assessment math algorithms in Javascript.

Step 4: Appointment Booking

- **What to Click**: Click **Get Appointment**. Fill out name, phone, date, choose hospital, doctor, and submit.
- **What to Say**: *"We are booking an appointment. The form is a clean multi-step wizard. When submitted, the backend inserts a patient profile and computes a queue token number dynamically using a secure SQL transaction."*
- **Demonstrated**: Multi-step booking form and database transactions.

Step 5: Blood Donation Registry

- **What to Click**: Click **Donate Blood**. Fill out donor registration form. Fill out the request blood form.
- **What to Say**: *"This is our blood network. Users register as donors, and hospitals log urgent blood requirements. The system keeps record of matching blood types."*
- **Demonstrated**: Blood bank donor registry.

Step 6: Log in as Hospital Admin

- **What to Click**: Click **Login** in navbar. Select **Hospital Admin**. Log in.
- **What to Say**: *"I am logging in as a Hospital Admin. The system verifies credentials using BCrypt password hashing. My dashboard is scoped to my hospital only, displaying statistics, pagination, and a live alert bell that polls the database for new notifications."*
- **Demonstrated**: BCrypt security, role-based dashboards, and live alerts polling.

Step 7: Log in as Doctor & Diagnosis

- **What to Click**: Log out. Log in as a **Doctor**. Click **Checkup** on the waiting patient. Enter prescription details and submit.
- **What to Say**: *"Logged in as a doctor. The dashboard displays the today's queue. I click Checkup, enter the diagnosis and medicines, and submit. The appointment status updates to completed and records the prescription."*
- **Demonstrated**: EMR checkup flow and EMR records saving.

Step 8: Print Prescription

- **What to Click**: Click the **Print** button next to the completed patient record.
- **What to Say**: *"The system generates an A4-optimized print view of the prescription, complete with patient details, token number, doctor name, and specialization, ready to be handed to the patient."*
- **Demonstrated**: Printing module.

---

10. Viva Questions and Answers (80 Q&As)

Project Idea & Problem Statement

1. **Q: What is the main theme of your project?**

- **A**: It is a multi-tenant SaaS healthcare platform designed to digitize appointments, medical records, and blood networks in regional areas.

2. **Q: Why choose a SaaS model for this?**

- **A**: It allows multiple hospitals and clinics to register and run their operations independently on a single platform, eliminating the cost of separate software installations.

3. **Q: What specific problem does it solve in regional areas?**

- **A**: It eliminates unorganized patient queues, automates blood donor matching during emergencies, and provides patient-level wellness tools.

4. **Q: How does the system handle multi-tenancy?**

- **A**: Most tables (doctors, appointments, schedules, notifications) contain a `hospital_id` foreign key. Upon login, queries are strictly scoped to the user's `hospital_id`.

5. **Q: Who are the direct stakeholders of this system?**

- **A**: Patients, clinic doctors, receptionists, hospital administrators, and the super administrator.

Architecture & Framework

6. **Q: Why did you build a custom MVC framework in raw PHP instead of using Laravel?**

- **A**: To avoid heavy framework overhead, ensure maximum performance on low-resource hosting, and demonstrate a deep understanding of OOP design patterns and routing to the evaluation panel.

7. **Q: Explain what MVC is.**

- **A**: Model-View-Controller. The *Model* manages database tables and business logic. The *View* handles the presentation layer (HTML templates). The *Controller* binds them together by processing inputs, updating models, and rendering views.

8. **Q: What is the role of `index.php` in the backend?**

- **A**: It is the single entry point (Front Controller). It starts sessions, imports config files, and initializes the Router.

9. **Q: How does `Router.php` work?**

- **A**: It reads the `route` query parameter (e.g., `?route=auth/login`), splits it, loads the corresponding controller file dynamically, instantiates the class, and calls the requested method.

10. **Q: How does the base `Controller` class help other controllers?**

- **A**: It holds the common database connection instance (`$this->db`) and exposes helper methods like `view()` to render templates and `redirect()` for URL routing.

Frontend Technologies

11. **Q: Why did you choose jQuery instead of modern libraries like React or Vue?**
    - **A**: jQuery provides fast DOM manipulation and AJAX calls that are easy to configure, lightweight, and compile directly in any browser without needing build tools (webpack/npm).
12. **Q: How does the frontend communicate with the backend?**
    - **A**: Via asynchronous `fetch()` API calls and jQuery `$.getJSON` or `$.ajax` calls pointing to `backend/index.php?route=api/...`.
13. **Q: How are animations handled on the landing page?**
    - **A**: Using standard CSS animations combined with `IntersectionObserver` in JavaScript to trigger CSS classes (like `is-visible`) when cards scroll into view.
14. **Q: What is "glassmorphism" in CSS?**
    - **A**: A design style that uses semi-transparent backgrounds, blur filters (`backdrop-filter: blur()`), and subtle borders to create a frosted-glass effect.
15. **Q: How do you handle responsive design?**
    - **A**: By combining Bootstrap 5's responsive grid system (using classes like `col-md-6`, `col-lg-4`) with custom media queries (`@media`) for fine styling adjustments on mobile devices.

Database Design & SQL

16. **Q: What database engine did you select and why?**
    - **A**: MySQL/MariaDB because it is relational, highly optimized, free, and standard for local environments like XAMPP.
17. **Q: Explain what a foreign key is.**
    - **A**: A field in one table (e.g., `appointments.patient_id`) that links to the primary key of another table (`patients_details.id`), enforcing relational integrity.
18. **Q: What is a junction table? Give an example from your database.**
    - **A**: A table used to map a many-to-many relationship. An example is `doctor_hospital_affiliations`, which connects doctors to hospitals since a doctor can work at multiple hospitals and a hospital has many doctors.
19. **Q: Why do you use database transactions?**
    - **A**: To guarantee data integrity. In appointment booking, we register a user, insert their profile details, and save the appointment. If any insert fails, the transaction is rolled back (`rollBack()`) so no partial, orphan data remains.
20. **Q: What are the tablespaces `ibdata1` and `ibtmp1` in MySQL?**
    - **A**: `ibdata1` is the system tablespace for the InnoDB storage engine containing data dictionary metadata. `ibtmp1` holds temporary tables created during complex queries.

Session Management & Security

21. **Q: How are user sessions secured in PHP?**
    - **A**: Sessions are managed using PHP's built-in `session_start()`. Session IDs are stored in cookies. To prevent

hijackings, access checks are performed on every controller load.

22. **Q: How does role-based access control (RBAC) work in your code?**
    - **A**: Upon login, the user's role is stored in `$_SESSION['role']`. In the constructor of each controller, we check this value (e.g., `$_SESSION['role'] != 'hospital_admin'`) and block access if it does not match.
23. **Q: How do you secure user passwords?**
    - **A**: Passwords are hashed using BCrypt (`password_hash()` in PHP) before database insertion. During authentication, we verify them using `password_verify()`.
24. **Q: What is SQL Injection and how does your project prevent it?**
    - **A**: SQL Injection is a vulnerability where attackers inject malicious SQL queries into input fields. We prevent this by using PDO **prepared statements** and parameter binding (`bindParam()`), which treat inputs as data variables, not executable code.
25. **Q: How do you prevent Cross-Site Scripting (XSS) in views?**
    - **A**: By escaping all dynamic variables printed on the screen using `htmlspecialchars()` to turn HTML tags into plain text blocks.

Appointment Booking

26. **Q: How is the token number calculated for appointments?**
    - **A**: The system runs a `COUNT() query for appointments scheduled with the same doctor on the selected date. The next token number is this count plus one.`
27. **Q: What happens if two patients book the same doctor at the same time?**
    - **A**: Since bookings are date-scoped rather than specific minute-scoped, they both get queued for that day with unique token numbers (e.g., Token 5 and Token 6) to be checked in sequence.
28. **Q: How is today's appointment queue rendered on the doctor's dashboard?**
    - **A**: The dashboard controller calls `getTodayAppointments()` for the doctor's user ID. The view loops through the results and renders only today's list sorted by token number.
29. **Q: Why does the doctor dashboard show a Checkup button for both waiting and confirmed statuses?**
    - **A**: Because confirmed online bookings and walk-in waiting list patients are both ready to be diagnosed by the doctor.
30. **Q: What database status values does an appointment transition through?**
    - **A**: `pending` (online booking awaiting review) -> `confirmed`/`waiting` (approved/ready for checkup) -> `completed` (prescribed) or `cancelled`.

Health Assessments

31. **Q: Is the Health Assessment suite a replacement for clinical diagnosis?**
    - **A**: No, it is a self-service screening tool designed to raise wellness awareness. A disclaimer on the UI instructs users to consult a doctor for official medical advice.
32. **Q: What is the math formula behind the Calorie Calculator?**
    - **A**: The Mifflin-St Jeor equation. It computes Basal Metabolic Rate (BMR) based on age, gender, weight, and height,

and then applies an activity multiplier.

33. **Q: How do you verify blood donation eligibility?**
    - **A**: By validating weight (must be >= 50kg), age (between 18 and 65), last donation date (> 90 days ago), and safety questions.
34. **Q: How is the BMI score calculated in JavaScript?**
    - **A**: $BMI = \frac{weight}{(height/100)^2}$.
35. **Q: How are health results displayed without reloading the page?**
    - **A**: JavaScript intercepts the submit button click, calculates the values, modifies the text content of the result card, and fades it into view using CSS transitions.

Blood Donation Module

36. **Q: How do blood requests work in MedNova?**
    - **A**: A user inputs patient details, blood group, urgency, and the hospital location. The backend saves this and triggers an alert.
37. **Q: How are blood donation requests broadcasted?**
    - **A**: When saved, a notification is inserted into the `notifications` table, making it immediately visible to the corresponding hospital admin.
38. **Q: How does matching donors work?**
    - **A**: The admin dashboard queries the `blood_donors` table filtering for the requested blood group and hospital region.
39. **Q: Is there active donor verification?**
    - **A**: Yes, the system matches their contact details and eligibility status.
40. **Q: What details are stored for blood donors?**
    - **A**: User reference ID, blood type, age, phone number, location, and the affiliated hospital base.

Diet Plan Module

41. **Q: Who has the authority to create diet plans?**
    - **A**: Registered doctors. They create and assign the plans to patients.
42. **Q: How does a patient view their diet plan?**
    - **A**: They log into their patient dashboard, which queries the `diet_plans` table for records matching their user ID.
43. **Q: What values are saved in the `diet_plans` table?**
    - **A**: Patient ID, doctor ID, weight, height, age, target goal (loss/gain), meal schedules, and custom instructions.
44. **Q: Can a hospital admin delete diet plans?**
    - **A**: Yes, hospital admins have administrative rights to manage and clean local clinic records.

45. **Q: How is doctor-patient mapping enforced in diet plans?**
    - **A**: Through foreign keys mapping the patient's user ID and doctor's user ID to the `users` table.

Administration & SaaS

46. **Q: What is the main difference between Super Admin and Hospital Admin?**
    - **A**: Super Admin manages the global platform, subscriptions, and hospital accounts. Hospital Admin manages only the staff, doctors, and configurations of their own hospital.
47. **Q: How does subscription plan tracking work?**
    - **A**: The `hospitals` table has columns `subscription_status` (active/inactive), `plan_type`, and `plan_expires_at`. The system limits access if the expiration date passes.
48. **Q: How do you handle database backups in the admin panel?**
    - **A**: The admin controller runs a script that queries all database tables, generates `CREATE TABLE` and `INSERT INTO` SQL statements, and outputs them as a downloadable `.sql` file.
49. **Q: Can the administrator restore the database from a backup?**
    - **A**: Yes, the system reads the uploaded `.sql` file and executes the queries to restore the tables and data.
50. **Q: How do you display live alerts to the Hospital Admin?**
    - **A**: The navbar uses an AJAX polling function that queries the server every few seconds for new records in the `notifications` table since the last check.

Receptionist & Walk-in Flow

51. **Q: Why is the receptionist role needed if patients can book online?**
    - **A**: For walk-in patients who arrive at the clinic without booking. The receptionist registers them and adds them to the queue instantly.
52. **Q: What happens when a receptionist registers a walk-in patient?**
    - **A**: The system creates a user profile, generates an appointment record with status `'waiting'`, and prints a physical token slip.
53. **Q: Where is receptionist-to-hospital mapping defined?**
    - **A**: In the `receptionist_hospital_affiliations` junction table.
54. **Q: How is the receptionist dashboard sorted?**
    - **A**: It shows today's appointments for their affiliated hospital, sorted by token number.
55. **Q: Can a receptionist change a patient's diagnosis?**
    - **A**: No, only the assigned doctor has the security privilege to write or modify prescriptions.

Diagnostic & EMR Flow

56. **Q: What happens during a patient checkup?**

- **A**: The doctor enters details in the diagnosis form (symptoms, medicines, advice). The backend inserts a record into `prescriptions` and marks the appointment as `completed`.

57. **Q: How is the prescription linked to the patient?**

- **A**: Through the appointment ID. The prescription table references the appointment, which in turn references the patient.

58. **Q: How does the A4 printing stylesheet work?**

- **A**: We use CSS print media queries (`@media print`) to hide navigation menus, sidebars, and buttons, displaying only the prescription layout.

59. **Q: What is saved in the `prescriptions` table?**

- **A**: Appointment ID, diagnosis, medicines, and advice.

60. **Q: Can doctors view patient medical history?**

- **A**: Yes, the system allows doctors to search and review past prescriptions linked to a patient's profile.

Error Handling & Reliability

61. **Q: What is database port probing?**

- **A**: On start, the database class attempts connection on port `3306`. If it fails, it tries alternative ports (`3308`, `3307`). This ensures connection success on different local configurations.

62. **Q: What happens if the database does not exist?**

- **A**: The database class catches PDO error code `1049` (unknown database), triggers `createDatabase()`, runs `CREATE DATABASE`, and imports the `schema.sql` template.

63. **Q: How do you prevent application hangs on database timeouts?**

- **A**: By setting a PDO timeout option: `PDO::ATTR_TIMEOUT => 2`.

64. **Q: How does the system handle division by zero errors in the BMI calculator?**

- **A**: The JavaScript code validates height input; if height is 0 or negative, it shows a warning and stops the calculation.

65. **Q: How does the API respond during database failures?**

- **A**: It catches exceptions inside a `try-catch` block and returns a `500 Internal Server Error` status with a JSON error message.

Testing & Quality Assurance

66. **Q: How did you test the functionality of your API routes?**

- **A**: By performing HTTP checks using curl and verifying JSON responses.

67. **Q: What is the purpose of the `check_links.php` script?**

- **A**: It recursively scans HTML and PHP files to identify broken paths and verify valid routing targets.

68. **Q: How did you test security restrictions?**

- **A**: By attempting to access admin and doctor routes (e.g., `?route=admin/dashboard`) from an unauthenticated browser session, confirming the system redirects users to the login screen.

69. **Q: What is unit testing and did you use it?**

- **A**: Unit testing verifies individual components. We validated modules by simulating inputs and verifying expected database states.

70. **Q: How did you test database load?**

- **A**: By running the database seeder script (`seed_dummy.php`) to generate hundreds of mock records across all tables.

Future Enhancements & Scalability

71. **Q: How would you scale this application to support millions of users?**

- **A**: Move the database to a managed cloud cluster, configure read-replicas, use a CDN for frontend assets, and implement redis caching for session data.

72. **Q: How can you integrate real-time mobile alerts?**

- **A**: By integrating SMS gateways (like Twilio) inside the notification triggers in `ApiController.php`.

73. **Q: How would you add AI report scanning?**

- **A**: Implement an Optical Character Recognition (OCR) API (like Tesseract or Google Cloud Vision) to parse uploaded report images, and use an LLM API to summarize values.

74. **Q: Can this be converted into a mobile app?**

- **A**: Yes, the backend is built on RESTful JSON APIs, meaning you can build React Native or Flutter mobile apps that connect to the same endpoints.

75. **Q: How will you handle payment processing?**

- **A**: By integrating payment gateways (like Stripe or PayPal) within the appointment confirmation workflow.

Miscellaneous

76. **Q: What does the `password_hash()` default algorithm use?**

- **A**: Currently BCrypt, which is a secure, resource-adaptive hashing algorithm.

77. **: How do you prevent double form submissions?**

- **A**: By disabling the submit button (`prop('disabled', true)`) in jQuery immediately after submission begins.

78. **Q: Where is the app logo stored?**

- **A**: In `frontend/assets/logo.jpeg`.

79. **Q: What does "SaaS" mean?**

- **A**: Software-as-a-Service, where the software is hosted centrally and licensed on a subscription basis.

80. **Q: What is the main conclusion of your project?**

- **A**: MedNova successfully digitizes clinical workflows, queues, and blood donor networks, providing a practical SaaS

solution for regional clinics.

11. Difficult Examiner Questions & Strong Defenses

1. "Why did you use raw PHP instead of a modern framework like Laravel?"

- **Defense**: *"While Laravel is excellent for large enterprise applications, it comes with significant boilerplate, dependencies, and resource overhead. For regional clinics and shared hosting environments, raw PHP executes significantly faster and has a minimal memory footprint. More importantly, building our own routing engine and MVC pattern allows us to demonstrate a deep understanding of core web architecture, OOP principles, and clean database interactions to this panel, rather than relying on automated framework components."*

2. "How secure is patient data in your system?"

- **Defense**: *"Patient data security is enforced at three levels:*

1. **Data Ingestion**: We prevent SQL injections by using PDO prepared statements and parameter binding.
2. **Authentication**: User passwords are saved as secure BCrypt hashes; clear-text passwords are never stored.
3. **Session Guards**: Every request is validated against the active session role. If a patient attempts to access a doctor or admin route, they are blocked immediately. In a production build, we would also enforce HTTPS to encrypt data in transit."

3. "Why MySQL? Why not a NoSQL database like MongoDB?"

- **Defense**: *"Healthcare data is highly relational. A patient has appointments, appointments have prescriptions, and doctors belong to specific hospitals. Relational databases like MySQL enforce referential integrity and data consistency using strict foreign keys. Furthermore, MySQL supports ACID transactions, ensuring that multi-table inserts (like user registration + patient profile creation + appointment token assignment) either succeed completely or fail completely, preventing orphan records."*

4. "Is your Health Assessment suite a medical diagnosis tool?"

- **Defense**: *"No, it is strictly a wellness and screening tool. All algorithms (BMI, Mifflin-St Jeor, and water targets) are based on established general wellness formulas. The user interface includes clear disclaimers stating that the results are for educational purposes only and cannot substitute for professional medical consultations."*

5. "What happens if the local database port is not 3306?"

- **Defense**: *"Our system features database connection resilience. On startup, the Database class probes port `3306`. If it fails, it automatically probes alternative ports `3308` and `3307`. This ensures compatibility with custom local environments without requiring manual edits to database configuration files."*

6. "How does the system handle session hijacking?"

- **Defense**: *"We use standard PHP session identifiers. To enhance security in a production deployment, we would configure secure session cookie flags (`HttpOnly` and `Secure`) to prevent JavaScript access and ensure session cookies are only transmitted over encrypted HTTPS connections."*

7. "Why did you use jQuery instead of React?"

- **Defense**: *"React is highly suited for complex, state-heavy single-page applications, but it requires a compilation build step (npm, webpack) and increases initial page load times due to bundle sizes. Since MedNova is a SaaS portal*

intended to load quickly on regional, low-bandwidth connections, lightweight HTML pages coupled with jQuery for asynchronous AJAX requests provide a fast, compilation-free frontend experience."

8. "How does the system prevent double bookings for the same doctor?"

- **Defense**: *"We use a queue-based token system. Patients book an appointment for a specific date. During booking, the system queries existing bookings for that doctor and date inside a database transaction, assigning the next sequential token number. This places patients in a clean sequence, eliminating scheduling conflicts."*

9. "What will happen if the internet connection is lost?"

- **Defense**: *"Since the core SaaS features operate online, an internet connection is required to communicate with the database. For local usage inside clinics, MedNova can be hosted on a local intranet server (via XAMPP) within the hospital's local network, allowing full functionality even without external internet access."*

10. "How do you handle scaling as more hospitals register?"

- **Defense**: *"Because the architecture is modular and uses a normalized relational database, we can scale vertically by hosting the system on managed cloud servers (like AWS or DigitalOcean) and database clusters, separating read and write queries."*

11. "Why do you generate a dummy email for patients who register with phone numbers?"

- **Defense**: *"To simplify registration. Patients booking appointments only need to provide their name, phone, and date. To ensure database integrity (where `users.email` is a unique field), the system auto-generates a unique temporary email format based on their phone number and timestamp. The patient can update this to their real email later in their profile settings."*

12. "What are the limitations of the BMI calculator?"

- **Defense**: *"The BMI calculator only uses height and weight. It does not distinguish between muscle mass and fat mass, making it less accurate for athletes or bodybuilders. The UI notes this limitation, advising users to look at body fat percentage and waist circumference for a more complete picture."*

13. "What happens to notifications older than 48 hours?"

- **Defense**: *"Our query filters notifications from the last 48 hours for display in the navbar dropdown, preventing performance issues. Older notifications remain stored in the database and can be reviewed in the comprehensive notifications view page."*

14. "How do you prevent brute force attacks on the login page?"

- **Defense**: *"In this release, login credentials are validated securely on the server. In a production build, we would implement rate-limiting by tracking failed login attempts from specific IP addresses, locking accounts or IP ranges after multiple failures."*

15. "Why isn't there an online payment gateway integrated?"

- **Defense**: *"This version focuses on operational scheduling and record management. We have laid the database groundwork with a `payments` table. Integrating live payment processors (like Stripe or PayPal APIs) is planned as a future upgrade."*

16. "How will you verify that a blood donor is safe to donate?"

- **Defense**: *"The system performs a basic eligibility check based on user-entered parameters (age, weight, donation history). The final clinical validation is performed by medical staff at the physical clinic before the donation takes place."*

17. "What is database normalization and is your database normalized?"

- **Defense**: *"Yes, our database is normalized up to the Third Normal Form (3NF). We have separated user credentials, doctor specialty details, and hospital metadata into distinct tables, mapping relationships through foreign keys. This minimizes data redundancy and prevents update anomalies."*

18. "Why does the base Controller class instantiate PHPMailer?"

- **Defense**: *"To support automatic email notifications (such as appointment confirmations or password resets). PHPMailer is integrated in the base class so that any inheriting controller can trigger SMTP emails using a single method call."*

19. "How secure is the database backup module?"

- **Defense**: *"The backup module in the Admin Panel is restricted to the Super Admin role. The script generates a raw SQL structure. To secure this in production, backups are encrypted and stored in private cloud storage."*

20. "Why did you use Mifflin-St Jeor instead of Harris-Benedict for calories?"

- **Defense**: *"The Mifflin-St Jeor equation is recognized as more accurate for modern populations, particularly when calculating basal metabolic rates."*

21. "What is the purpose of the `doctor_hospital_affiliations` table?"

- **Defense**: *"It maps the many-to-many relationship between doctors and hospitals, allowing a doctor to be affiliated with multiple clinics, and a clinic to host multiple doctors."*

22. "How does the system calculate water intake?"

- **Defense**: *"By multiplying the user's weight in kilograms by 35ml, and adding 500ml for each hour of daily exercise."*

23. "Why does the system run database checks on startup?"

- **Defense**: *"To simplify deployment. When setting up on a new server, the database class automatically creates the tables and imports the initial schema from `schema.sql` if they are missing."*

24. "How do you prevent Cross-Site Request Forgery (CSRF)?"

- **Defense**: *"We validate active sessions on all state-changing requests. For production, we will implement unique CSRF tokens generated per session and verified on form submissions."*

25. "Why does the receptionist dashboard show print tokens?"

- **Defense**: *"So that walk-in patients can receive a physical token slip with their queue number and doctor details upon registration."*

26. "What happens if a doctor is deleted?"

- **Defense**: *"We delete the doctor's record from the database. In production, we would use soft deletes (setting an `is_deleted` flag to `1`) to preserve historical prescription records."*

27. "How do you structure CSS styles?"

- **Defense**: *"We use a custom style sheet with CSS variables for colors, fonts, and spacing, ensuring a consistent theme across the application."*

28. "How does the patient dashboard retrieve history?"

- **A**: *"By querying the appointments table for records matching the logged-in patient's user ID, sorted by date."*

29. "What will happen if the schema.sql file is missing on setup?"

- **Defense**: *"The system will fail to auto-initialize the database structure. The `schema.sql` file must reside in the root directory for the automated setup script to run successfully."*

30. "What was the most challenging part of this project?"

- **Defense**: *"Designing a custom MVC routing engine and handling database connection port probing on local environments, ensuring the application remains portable."*

12. Team Role Explanation & Presentation Strategy

For a successful presentation, divide responsibilities within your team as follows:

12.1 Frontend Member

- **Responsibility**: Present the client portal, user interface, responsive layout, health calculators, and CSS/JS animations.
- **Questions to Expect**: *"How does jQuery calculate calories?", "Why did you use Bootstrap?"*
- **Best Response**: Focus on user experience, responsive design, fast client-side calculations, and the single-page application feel of the interface.

12.2 Backend Member

- **Responsibility**: Explain the MVC architecture, custom routing engine, controllers, controllers authentication, session checks, and SMTP integrations.
- **Questions to Expect**: *"How does the router parse requests?", "Why not use Laravel?"*
- **Best Response**: Highlight code performance, lightweight execution, security layers, and how controllers bind models to views.

12.3 Database Member

- **Responsibility**: Explain database normalization, entity relationships (PK/FK), transactions, and connection port probing.
- **Questions to Expect**: *"Why do you need transactions?", "Explain the junction tables."*
- **Best Response**: Focus on data consistency, ACID compliance, preventing data corruption, and connection resilience.

12.4 Quality Assurance & Documentation Member

- **Responsibility**: Discuss testing strategies (linting, link checks, security checks), system limitations, and future improvements.
- **Questions to Expect**: *"How did you test for broken links?", "What are the system limitations?"*
- **Best Response**: Explain systematic testing methodologies, validation results, and clear technical pathways for future upgrades.

13. Code Understanding Guide

Quickly locate specific code blocks when asked by examiners:

- **Database Connections & Port Probing**:
- File: [Database.php](file:///c:/Users/Munim%20Abbas/Desktop/FinalFYP/backend/config/Database.php) (lines 40-118).
- **MVC Routing Engine**:
- File: [Router.php](file:///c:/Users/Munim%20Abbas/Desktop/FinalFYP/backend/core/Router.php) (lines 20-58).
- **Base Controller View Loader**:
- File: [Controller.php](file:///c:/Users/Munim%20Abbas/Desktop/FinalFYP/backend/core/Controller.php) (lines 25-35).
- **Appointment Booking Transaction**:
- File: [ApiController.php](file:///c:/Users/Munim%20Abbas/Desktop/FinalFYP/backend/controllers/ApiController.php) (lines 237-287).
- **Client Asynchronous API Requests**:
- File: [app.js](file:///c:/Users/Munim%20Abbas/Desktop/FinalFYP/frontend/js/app.js) (lines 35-120).
- **Health Calculator Formulas**:
- File: [health-assessment.html](file:///c:/Users/Munim%20Abbas/Desktop/FinalFYP/frontend/health-assessment.html) (script section at bottom).
- **Blood Request Notifications Broadcast**:
- File: [ApiController.php](file:///c:/Users/Munim%20Abbas/Desktop/FinalFYP/backend/controllers/ApiController.php) (lines 380-410).
- **Doctor Dashboard Queue View**:
- File: [dashboard.php](file:///c:/Users/Munim%20Abbas/Desktop/FinalFYP/backend/views/doctor/dashboard.php) (lines 20-49).

14. Limitations and Future Enhancements

14.1 Current Limitations

- **Notification Scope**: Notifications use standard client-side database polling rather than web sockets.
- **Payment Gateway**: Relies on manual payment logging instead of direct online processing.
- **SMS Reminders**: Email notifications are supported via PHPMailer, but SMS notifications require an active external gateway subscription.

14.2 Future Work

- **Web Sockets Integration**: Switch to real-time notification alerts using socket.io.
- **Optical Character Recognition (OCR)**: Let patients upload lab reports directly, parsing values automatically using machine learning.
- **Mobile App Development**: Build mobile apps (iOS and Android) that connect to MedNova's JSON API endpoints.

15. Final Defense Cheat Sheet

- **Project Title**: MedNova Smart Digital Healthcare Platform
- **Problem Statement**: Fragmented medical operations and manual patient queuing in regional clinic networks.
- **Architecture**: Relational database, raw PHP MVC architecture, asynchronous jQuery frontend.
- **Strongest Points**:

1. **Framework-Free MVC**: Showcases clean, fundamental coding structures.
2. **ACID Database Transactions**: Guarantees secure data updates.
3. **Connection Resilience**: Probes database ports automatically on setup.

- **Golden Presentation Rule**: *"Be confident, emphasize that you built the system from scratch to learn core web architecture, and reference your code locations directly."*